

Software Requirement Specification (SRS)

Project:	Chief of Staff AI (The Draft Desk)
Version:	1.0
Date:	July 26, 2026
Author:	V Sree Kirthana
Role:	AI Specialist & Full-Stack Developer

1. Introduction

1.1 Purpose

The purpose of this document is to define the system architecture, features, and requirements for Chief of Staff AI (The Draft Desk). This intelligent agent acts as an autonomous gatekeeper for a user's inbox, leveraging advanced LLMs to read, classify, and draft responses to emails, while providing seamless calendar scheduling capabilities.

1.2 Scope

The Chief of Staff AI integrates directly with the user's Google Workspace (Gmail and Google Calendar). It operates on a strict, transparent four-step pipeline: Inbox & Triage, Draft Generation, Approval Gate, and Export Proof. The system emphasizes a "Human-in-the-Loop" architecture, ensuring no automated actions (sending emails or booking meetings) occur without explicit user consent.

2. System Architecture & Tech Stack

2.1 Technology Stack

- **Core Logic:** Python 3.x
- **Frontend UI:** Streamlit (Custom Aurora Glassmorphism framework)
- **AI Engine:** Google Gemini 2.5 Flash
- **Cloud Infrastructure:** Google Cloud Platform (GCP)
- **Authentication:** OAuth 2.0 Web Flow (PKCE)
- **External Integrations:** Gmail API, Google Calendar API

2.2 Development Environment

- **IDE:** Visual Studio Code (VS Code)
- **Version Control:** Git & GitHub
- **Local Server:** Streamlit local host
- **Deployment Platform:** Streamlit Community Cloud

2.3 Environment Setup Requirements

To replicate or deploy the environment, the following configurations are strictly required:

- **Google Cloud Console:** A registered GCP project with the Gmail API and Google Calendar API enabled.
- **OAuth Credentials:** Configured OAuth consent screen and generated `credentials.json` (or Streamlit secrets equivalent) with scopes for `gmail.readonly`, `gmail.send`, and `calendar`.
- **Dependencies:** `requirements.txt` containing `streamlit`, `google-auth-oauthlib`, `google-api-python-client`, and Gemini AI SDKs.
- **Local Execution:** Executed via `streamlit run app.py` with `OAUTHLIB_INSECURE_TRANSPORT="1"` configured for local OAuth testing.

3. Functional Requirements

3.1 Authentication & Security Flow

- **FR-1.1:** The system shall authenticate users via Google OAuth 2.0 Web Flow.
- **FR-1.2:** The system shall restrict access to the main dashboard until a valid OAuth token is retrieved and verified.

3.2 Phase 1: Inbox & Triage

- **FR-2.1:** The system shall fetch live email threads from the authenticated user's Gmail inbox.
- **FR-2.2:** The AI engine shall analyze the context, intent, and urgency of every message.
- **FR-2.3:** The system shall classify emails into four distinct categories: Urgent (●), Needs Reply (●), FYI (●), and Ignore (●).

3.3 Phase 2: Draft Generation

- **FR-3.1:** The system shall automatically generate highly contextual, professional draft responses exclusively for emails flagged as Urgent or Needs Reply.
- **FR-3.2:** The system shall handle API rate limits gracefully, displaying a visual queue/warning if the Gemini API quota is exhausted.

3.4 Phase 3: Approval Gate (Human-in-the-Loop)

- **FR-4.1:** The system shall display original email threads side-by-side with the AI-generated drafts.
- **FR-4.2:** The user shall have three actionable controls for every draft: Regenerate, Edit, and Approve.
- **FR-4.3:** The system shall dispatch emails directly to the recipient via the Gmail API only after the user clicks "Send" on an approved draft.

3.5 Smart Calendar Scheduling

- **FR-5.1:** The AI shall parse incoming emails for meeting requests, proposed times, and durations.
- **FR-5.2:** The system shall cross-reference proposed times with the user's availability via the Google Calendar API.
- **FR-5.3:** Upon user approval, the system shall autonomously create a calendar event, add attendees, and insert the event link into the dashboard.

3.6 Phase 4: Export Proof

- **FR-6.1:** The system shall maintain an audit log of all triaged emails, approved drafts, and taken actions (Sent/Booked).
- **FR-6.2:** The user shall be able to export a "Proof of Work" report as a cleanly formatted Markdown (.md) or HTML (.html) file.

4. Non-Functional Requirements

4.1 UI/UX & Design (Usability)

- **NFR-1.1:** The landing page must feature a custom Aurora gradient background with a pure CSS "extra glitter" effect.
- **NFR-1.2:** UI components must utilize a "White Glassmorphism" aesthetic for high contrast and modern readability.

- **NFR-1.3:** The application must maintain a strict visual hierarchy, prioritizing typography.

4.2 Security

- **NFR-2.1:** The application shall not store user email data permanently. All thread fetching and drafting must remain in the active session state.
- **NFR-2.2:** All API calls to Google Cloud must be secured via short-lived OAuth tokens utilizing PKCE verifiers.

4.3 Performance & Reliability

- **NFR-3.1:** The AI drafting pipeline must process a queue of standard actionable emails seamlessly.
- **NFR-3.2:** The system must implement robust exception handling for API timeouts and quota limits, ensuring the app does not crash during external service outages.

5. Project Timeline & Delivery

Phase	Milestone	Status
Phase 1	Conceptualization & Architecture Design	Completed
Phase 2	Backend Integration (Gmail/Calendar APIs, OAuth setup)	Completed
Phase 3	AI Engine Integration (Gemini 2.5 Prompting & Triage logic)	Completed
Phase 4	Frontend Development (Glassmorphism UI, Streamlit State management)	Completed
Phase 5	Testing, Debugging, and Final Deployment	Completed

Total Development Lifecycle: 2 Weeks

Final Status: Production-Ready & Deployed